

Souvenirs

Amaru está comprando recuerdos en una tienda extranjera. Hay N **tipos** de recuerdos. En la tienda hay una cantidad infinita de recuerdos de cada tipo disponibles.

Cada tipo de recuerdo tiene un precio fijo. Es decir, un recuerdo de tipo i ($0 \leq i < N$) tiene un precio de $P[i]$ monedas, donde $P[i]$ es un entero positivo.

Amaru conoce que los tipos de recuerdos están ordenados en orden decreciente por su precio, y que los precios de todos los recuerdos son distintos. Específicamente, $P[0] > P[1] > \dots > P[N-1] > 0$. Además, él conoce el valor de $P[0]$. Desafortunadamente, Amaru no tiene ninguna otra información sobre los precios de los recuerdos.

Para comprar algunos recuerdos, Amaru realizará un número de transacciones con el vendedor.

Cada transacción consiste de los siguientes pasos:

1. Amaru entrega una cantidad (positiva) de monedas al vendedor.
2. El vendedor pone estas monedas en una pila sobre una mesa en la habitación de atrás, donde Amaru no puede verlas.
3. El vendedor considera cada tipo de recuerdo $0, 1, \dots, N-1$ en ese orden, uno por uno. Cada tipo es considerado **exactamente una vez** por transacción.
 - Cuando considera el recuerdo de tipo i , si el número actual de monedas en la pila es al menos $P[i]$, entonces
 - el vendedor retira $P[i]$ monedas de la pila, y
 - el vendedor pone un recuerdo de tipo i sobre la mesa.
4. El vendedor le devuelve a Amaru todas las monedas que quedan en la pila y todos los recuerdos que hay sobre la mesa.

Note que no hay monedas o recuerdos en la mesa antes de que comiencen las transacciones.

Su tarea es ordenarle a Amaru que realice una cierta cantidad de transacciones, tal que:

- en cada transacción compre **al menos un** recuerdo, y
- en total compre **exactamente** i recuerdos de tipo i para cada i tal que $0 \leq i < N$. Note que esto significa que Amaru no debe comprar ningún recuerdo de tipo 0.

Amaru no tiene que minimizar el número de transacciones y tiene una cantidad ilimitada de monedas disponibles.

Detalles de Implementación

Debe implementar la siguiente función.

```
void buy_souvenirs(int N, long long P0)
```

- N : el número de tipos de recuerdos.
- $P0$: el valor de $P[0]$.
- Esta función es llamada exactamente una vez por cada caso de prueba.

La función anterior puede realizar llamadas a la siguiente función para ordenarle a Amaru a hacer una transacción:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : el número de monedas que Amaru entrega al vendedor.
- La función retorna un par. El primer elemento del par es un arreglo L , que contiene los tipos de recuerdos que fueron comprados (en orden creciente). El segundo elemento es un entero R , el número de monedas que recibe Amaru después de la transacción.
- Se requiere que $P[0] > M \geq P[N - 1]$. La condición $P[0] > M$ asegura que Amaru no comprará ningún recuerdo de tipo 0, y $M \geq P[N - 1]$ asegura que Amaru comprará al menos un recuerdo. Si estas condiciones no se cumplen, su solución recibirá el veredicto `Output isn't correct: Invalid argument`. Note que a diferencia de $P[0]$, el valor de $P[N - 1]$ no se le da en la entrada.
- La función puede ser llamada a lo más 5000 veces en cada caso de prueba.

El funcionamiento del evaluador es **no adaptativo**. Esto significa que la secuencia de precios P está fijada antes que `buy_souvenirs` sea llamada.

Restricciones

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i tal que $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtareas

Subtarea	Puntos	Restricciones adicionales
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.
6	33	Sin restricciones adicionales.

Ejemplos

Considere la siguiente llamada:

```
buy_souvenirs(3, 4)
```

Hay $N = 3$ tipos de recuerdos y $P[0] = 4$. Observe que hay solo tres posibles secuencias de precios P : $[4, 3, 2]$, $[4, 3, 1]$, y $[4, 2, 1]$.

Asuma que `buy_souvenirs` llama a `transaction(2)`. Suponga que la llamada retorna $([2], 1)$, que significa que Amaru compró un recuerdo de tipo 2 y el vendedor le devolvió 1 moneda. Observe que esto permite deducir que $P = [4, 3, 1]$, dado que:

- Para $P = [4, 3, 2]$, `transaction(2)` debería retornar $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` debería retornar $([1], 0)$.

Luego `buy_souvenirs` puede llamar `transaction(3)`, que retorna $([1], 0)$, que significa que Amaru compró un recuerdo de tipo 1 y el vendedor le devuelve 0 monedas. Hasta ahora, en total, él ha comprado un recuerdo de tipo 1 y un recuerdo de tipo 2.

Finalmente, `buy_souvenirs` puede llamar a `transaction(1)`, que retorna $([2], 0)$, lo que significa que Amaru compró un recuerdo de tipo 2. Note que también se pudo haber usado `transaction(2)` aquí. En este punto, en total Amaru tiene un recuerdo de tipo 1 y dos recuerdos de tipo 2, como se requería.

Evaluador de ejemplo

Formato de entrada:

```
N
P[0]  P[1]  ... P[N-1]
```

Formato de salida:

```
Q[0]  Q[1]  ... Q[N-1]
```

Aquí $Q[i]$ es el número de recuerdos de tipo i comprados en total para cada i tal que $0 \leq i < N$.